Key-Value Stores

Course 5

Key-Value Stores: The Simplest Model

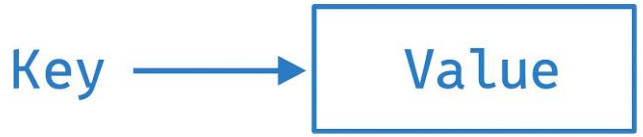
The foundational data model of NoSQL.

Essentially a massive, distributed **hash map**.

Optimized strictly for two operations: **GET(key)** and **PUT(key, value)**.

Structure

- **Key:** Unique identifier (string, hash). Used to locate the data.
- **Value:** An opaque blob of data. Can be JSON, an image, a string, etc.
- **Data Opacity:** The database does not inspect or care about the value's schema or structure.




```
{
  "user:123": { "name": "Alice", "age": 30 },
  "user:124": { "name": "Bob", "age": 25 },
  "session:456": "active"
}
```

Key

Is a unique identifier that acts like an address for retrieving the associated value.

Properties:

- Must be unique across the dataset.
- Typically immutable (changing a key means inserting a new pair).
- Often a string, but can be binary data depending on the database.
- Indexed efficiently (usually through hashing or sorted trees).

Example:

"user:123" or "session:abcdef123456"

Value

Is the actual data stored and retrieved via the key.

Properties:

- Can be any arbitrary blob of data — string, JSON object, image, serialized object, etc.
- The database doesn’t interpret or enforce structure (see data opacity on next slide).
- Often stored as a byte array internally.

Example:

Simple: "active"

Complex: { "name": "Alice", "email": "alice@example.com" }

Binary: a JPEG image file stored as bytes

Data Opacity

Means it treats values as opaque blobs - the database doesn't know or care what's inside them:

- The database cannot query inside the value (no "find users where age > 30").

- It only stores, retrieves, or deletes data based on the key.

- The application layer is responsible for:
    - Interpreting the value's structure
    - Managing serialization/deserialization (e.g., JSON, protobuf)

- Performing logical operations on the contents

Example:

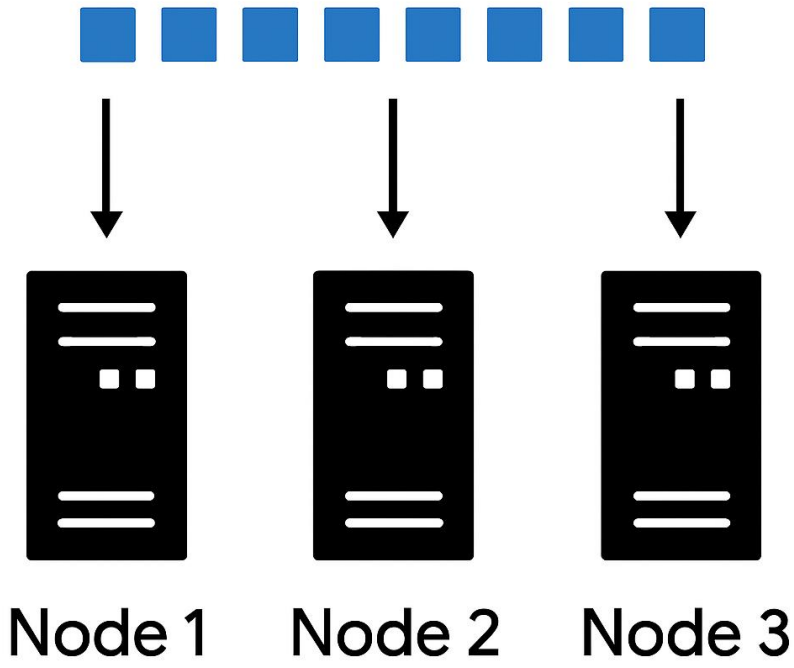
If the value is a JSON object:

{ "name": "Alice", "age": 30 }

The key–value store doesn't know about "name" or "age"; it just stores the entire JSON blob under "user:123".

Performance: O(1) Access Time

- **Speed:** Lookups are typically **O(1)** (constant time) because the key directly maps to the storage location.
- **CAP Model:** Generally **AP** (Available and Partition Tolerant), prioritizing availability.
- **Consistency:** Implements the **BASE** consistency model.
- **Scaling:** Linear and trivial **Horizontal Scaling** (add more nodes)



Internal Implementation

Hash Tables:

- Most in-memory KV stores (Redis) use hash tables for O(1) average access.

Persistent Stores:

- Disk-backed KV stores (LevelDB, RocksDB) use:
  - LSM Trees (Log-Structured Merge Trees): optimized for write-heavy workloads.
  - B-Trees / B+ Trees: used in older or more general-purpose databases.

Hash Table

A hash table maps keys to values using a hash function, which converts the key into an index (or "bucket") in an array.

Used as core data structure by in-memory key–value stores like Redis and Memcached.

Each key is hashed to an integer index:

hash("user:1") -> 3

hash("user:2") -> 7

hash("user:3") -> 3

The index decides where in the table to store the pair.

If two keys hash to the same index (a collision), uses a linked list or open addressing to handle it.

LSM Tree



An LSM Tree (Log-Structured Merge Tree) is optimized for high write throughput on disk.



Used by key–value stores like LevelDB, RocksDB, Cassandra, and DynamoDB.



Instead of updating records in place (which is slow on disks), it:

Writes new data sequentially to a log and an in-memory table (MemTable).

When the MemTable fills up, it is flushed to disk as an immutable sorted file (SSTable).

Over time, SSTables are merged (compacted) to keep lookups efficient.

1: Write to WAL (Write-Ahead Log)
    Appends to log for durability: WAL -> [("A",1), ("C",3), ("B",2)]
2: Insert into MemTable (in-memory sorted tree)
        MemTable -> { A:1, B:2, C:3 }
3: When MemTable is full -> Flush to SSTable
        SSTable_1 (on disk):
            A → 1
            B → 2
            C → 3
4: New writes go into a new MemTable, repeating the process.

When performing GET("B"):
    Search MemTable first (in memory)
    If not found, check immutable MemTables (read-only)
    If still not found, check SSTables on disk (using a bloom filter + binary search)

Common Use Cases

- **Caching Layer:** Stores frequently accessed query results (e.g., Redis).
- **Session Management:** Stores temporary user session data (login status, cart).
- **Rate Limiting:** Tracks API calls associated with a user or IP address.
- **Feature Toggles:** Stores global configuration settings for instant read access.

Data Distribution

Replication:

- Multiple copies of data on different nodes.
- Ensures fault tolerance, high availability, and read scalability.

Replication Models:

- Master–Slave: One node handles writes; others replicate.
- Multi-Master: All nodes can handle writes (eventual consistency).

Sharding (Partitioning):

- Splitting data across nodes based on key ranges or hashes.
- Common method: Consistent Hashing — evenly distributes keys, minimizes data movement on node changes.

Safe Parallel Access

Concurrency Control: Prevents race conditions during concurrent reads/writes.

- Optimistic concurrency: Allow concurrent ops, detect conflicts (e.g., version check).
- Pessimistic concurrency: Lock data during update.

Atomicity:

- Guarantees that an operation is all-or-nothing.
- Example: INCR key in Redis -> atomic counter update.

Isolation:

- Ensures consistent reads (e.g., Redis is single-threaded per instance -> implicit isolation).

Persistence and Recovery

Persistence:

- Ensures data survives crashes or restarts.
- Achieved using:
  - Write-Ahead Log (WAL): Append-only log of updates before applying them.
  - Snapshots: Periodic dumps of in-memory state to disk (e.g., Redis RDB).

Recovery:

- On restart, replay WAL + load latest snapshot to restore state.

Conflict Resolution

Why conflicts happen:

- Concurrent writes on different replicas -> inconsistent state.

Dynamo's Approach:

- Vector Clocks: Track version history of each object.
- Eventual Consistency: Allow temporary divergence, resolve later.
- Conflict Resolution Strategies:
  - Last Write Wins (LWW): Based on timestamp.
  - Read Repair: Resolve conflicts during read.
  - Client-side merge: Application decides how to reconcile.

AWS DynamoDB

A fully managed, multi-region NoSQL database service from AWS.

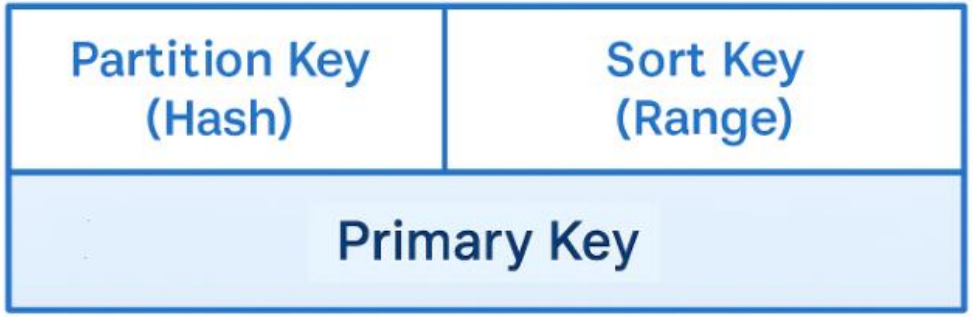
Supports both **Key-Value** and **Document** models.

Engineered for **single-digit millisecond performance** at any scale.

Offers **ACID Transactions** (unlike many traditional K/V systems).

Core Data Model

- DynamoDB's Primary Key is composed of up to two attributes:
  - **Partition Key (Hash Key):** Determines the data's physical location (shard).
  - **Sort Key (Range Key - Optional):** Defines the hierarchical order *within* that partition.



Partition Key: Sharding and Distribution



The Partition Key's hash value assigns data to a specific **physical partition (shard)**.



Must be designed for **high fan-out** to distribute read/write load evenly across partitions.

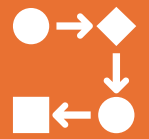


All queries **must** specify the Partition Key.



A poor PK choice leads to **"Hot Partitions"** (performance bottlenecks).

Sort Key: Hierarchy and Range Queries



Organizes items **sequentially** within a single partition (sorted by the Sort Key value).



Enables efficient **range queries** (e.g., "Get all customer orders between June and August").



Allows for data modeling patterns like **Adjacency Lists** within a single table.

Indexes

The Primary Key model is rigid (only fast lookups on PK/SK).

Indexes are separate data structures that allow querying on **non-key attributes**.

Allows the same data to be viewed and queried in multiple ways.

LSI: Strong Consistency within a Partition



Shares the **same Partition Key** as the base table.



Uses a **different Sort Key**.

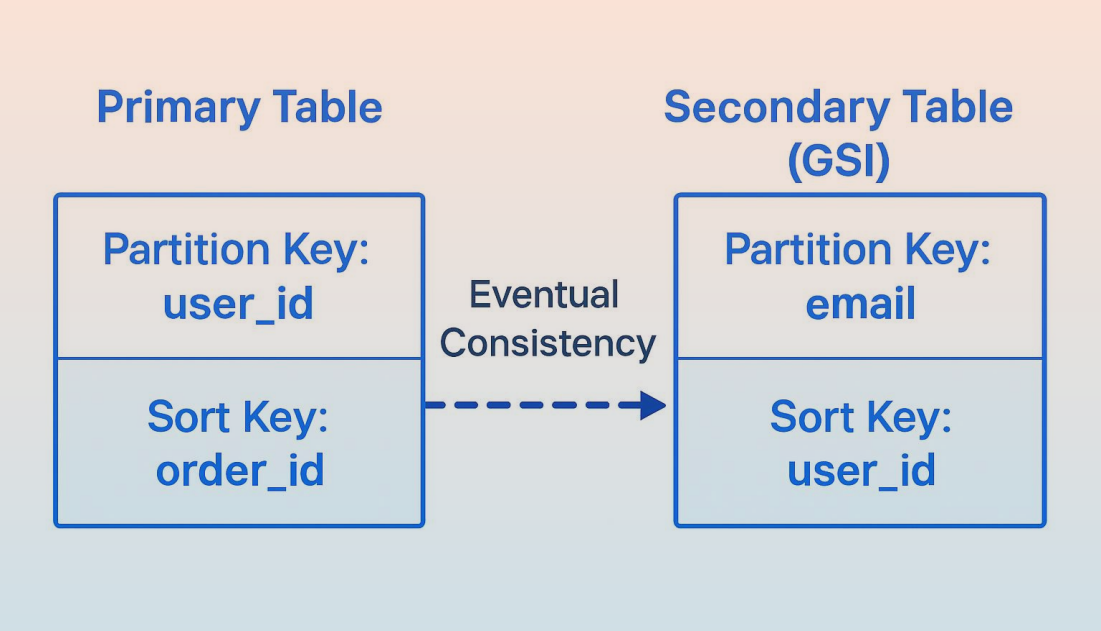


Reads are always **Strongly Consistent** (guaranteed latest data).



Confined to the limits of the original partition.

GSI: Global Flexibility and Scaling



Uses a **different Partition Key** (and optionally a different Sort Key) than the base table.

Ideal for high-fan-out queries (e.g., querying across all users by 'email').

Reads are always **Eventually Consistent** (most data, highest scale).

Can scale and be provisioned **independently** of the main table.

Capacity Modes: Provisioned vs. On-Demand

Provisioned:

- Pay for specified Read/Write Capacity Units (RCUs/WCUs) upfront.
- **Ideal for:** Predictable, steady workloads (cost certainty).
- **Risk:** Can lead to throttling if traffic exceeds provisioned limits.

On-Demand:

- Pay per million actual reads and writes executed.
- **Ideal for:** Spiky, unpredictable traffic (zero capacity planning).
- **Consistency:** Simplest operational model, handles peaks instantly.

Read Consistency: Performance vs. Freshness

Eventually Consistent Reads (Default): Fastest and most performant. A read might not reflect a very recent write.

Strongly Consistent Reads (Optional): Guarantees the latest, most up-to-date value. Consumes more capacity and has slightly higher latency.

ACID Guarantees with Transactions

DynamoDB supports **ACID** (Atomicity, Consistency, Isolation, Durability) transactions.

Allows multi-step updates across multiple items (records) within the same or different tables.

Use Case: Essential for financial or inventory workloads where strict correctness is mandatory.

Data Modeling: Access Patterns First



Anti-Pattern: Normalizing data and expecting joins.



Best Practice: Design the table structure (PK, SK, GSIs) to satisfy all application queries with **single, efficient lookups**.



Key Goal: Identify all access patterns and ensure a primary key or index can serve them.

Open Source KVs

- Redis – In-memory data store, often used for caching, session storage, and real-time analytics.
- Memcached – Simple, in-memory caching system for speeding up dynamic web applications.
- Riak KV – Distributed key-value database with fault tolerance and high availability.
- LevelDB – Embedded key-value store developed by Google; often used in local applications.
- RocksDB – High-performance embedded key-value store developed by Facebook, built on LevelDB.
- BadgerDB – Go-native key-value database inspired by RocksDB.

Commercial/Cloud KVs

- Amazon DynamoDB – Fully managed NoSQL key-value and document database by AWS.
- Azure Cosmos DB (Key-Value API) – Microsoft's globally distributed, multi-model database service.
- Google Cloud Datastore / Firestore (Key-Value mode) – Scalable, managed NoSQL database service.
- Couchbase – Distributed database that supports key-value and document data models.
- Aerospike – High-performance, low-latency distributed key-value store used in real-time systems.

Mastering DynamoDB



K/V: The fastest model, but requires keys for access.



DynamoDB: Uses the **Composite Key** (PK + SK) for core lookups.



GSIs: Crucial for flexibility and satisfying secondary access patterns.



Modeling: Denormalize data to avoid joins, design around your queries.

DynamoDB Core Components

- Tables – Similar to other database systems, DynamoDB stores data in tables. A table is a collection of data.

  Example: table called People can be used to store personal contact information about friends, family, or anyone else of interest. Also can have a Cars table to store information about vehicles that people drive.

- Items – Each table contains zero or more items. An item is a group of attributes that is uniquely identifiable among all of the other items.

  In a People table, each item represents a person. For a Cars table, each item represents one vehicle.

  Items in DynamoDB are similar in many ways to rows, records, or tuples in other database systems. In DynamoDB, there is no limit to the number of items you can store in a table.

- Attributes – Each item is composed of one or more attributes. An attribute is a fundamental data element, something that does not need to be broken down any further.

  An item in a People table contains attributes called PersonID, LastName, FirstName, and other. For a Department table, an item might have attributes such as DepartmentID, Name, Manager, and so on. Attributes in DynamoDB are similar in many ways to fields or columns in other database systems.

Primary Key

DynamoDB supports two different kinds of primary keys:

- **Partition key** – A simple primary key, composed of one attribute known as the partition key.

  Uses the partition key's value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored.

  In a table that has only a partition key, no two items can have the same partition key value.

  A People table is an example of a table with a simple primary key (PersonID). Access to any item in the People table can be done directly by providing the PersonId value for that item.

People

```
{
    "PersonID": 101,
    "LastName": "Smith",
    "FirstName": "Fred",
    "Phone": "555-4321"
}

{
    "PersonID": 102,
    "LastName": "Jones",
    "FirstName": "Mary",
    "Address": {
                "Street": "123 Main",
                "City": "Anytown",
                "State": "OH",
                "ZIPCode": 12345
    }
}

{
    "PersonID": 103,
    "LastName": "Stephens",
    "FirstName": "Howard",
    "Address": {
                "Street": "123 Main",
                "City": "London",
                "PostalCode": "ER3 5K8"
    },
    "FavoriteColor": "Blue"
}
```

Primary Key

DynamoDB supports two different kinds of primary keys:

- **Partition key and sort key** – Referred to as a composite primary key, this type of key is composed of two attributes. The first attribute is the partition key, and the second attribute is the sort key.

  Uses the partition key value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored. All items with the same partition key value are stored together, in sorted order by sort key value.

  In a table that has a partition key and a sort key, it's possible for multiple items to have the same partition key value. However, those items must have different sort key values.

  A Music table can have a composite primary key (Artist and SongTitle). Access to any item can be done directly in the Music table by providing the Artist and SongTitle values for that item.

Music


```
{
    "Artist": "No One You Know",
    "SongTitle": "My Dog Spot",
    "AlbumTitle": "Hey Now",
    "Price": 1.98,
    "Genre": "Country",
    "CriticRating": 8.4
}

{
    "Artist": "No One You Know",
    "SongTitle": "Somewhere Down The Road",
    "AlbumTitle": "Somewhat Famous",
    "Genre": "Country",
    "CriticRating": 8.4,
    "Year": 1984
}

{
    "Artist": "The Acme Band",
    "SongTitle": "Still in Love",
    "AlbumTitle": "The Buck Starts Here",
    "Price": 2.47,
    "Genre": "Rock",
    "PromotionInfo": {
        "RadioStationsPlaying": [
            "KHCR",
            "KQBX",
            "WTNR",
            "WJJH"
        ],
        "TourDates": {
            "Seattle": "20150622",
            "Cleveland": "20150630"
        },
        "Rotation": "Heavy"
    }
}
```

Primary Key

A composite primary key gives additional flexibility when querying data. By providing only the value for Artist, DynamoDB retrieves all of the songs by that artist. To retrieve only a subset of songs by a particular artist, can be used a value for Artist along with a range of values for SongTitle.

Secondary indexes

One or more secondary indexes can be created on a table. A secondary index allows querying the data in the table using an alternate key, in addition to queries against the primary key. DynamoDB doesn't require indexes, but they give applications more flexibility when querying data. After creating a secondary index on a table, data can be read from the index in the same way as reading from the table.

DynamoDB supports two kinds of indexes:

- Global secondary index – An index with a partition key and sort key that can be different from those on the table. The primary key values in global secondary indexes don't need to be unique.
- Local secondary index – An index that has the same partition key as the table, but a different sort key.

Secondary indexes

Global secondary indexes (GSIs) are indexes that span the entire table, allowing queries across all partition keys.

- Local secondary indexes (LSIs) are indexes that have the same partition key as the base table but a different sort key.
- Each table in DynamoDB has a quota of 20 global secondary indexes (default quota) and 5 local secondary indexes.

In the example Music table shown previously, you can query data items by Artist (partition key) or by Artist and SongTitle (partition key and sort key). What if you also wanted to query the data by Genre and AlbumTitle? To do this, you could create an index on Genre and AlbumTitle, and then query the index in much the same way as you'd query the Music table.

Consistency

- Local Secondary Index (LSI)

The LSI lives in the same partition as the base item.

When data is read from an LSI, DynamoDB can read from the same storage node as the base table.

Therefore, it supports both:

  - Strongly consistent reads (always reflects the latest write)
  - Eventually consistent reads (faster, cheaper)

- Global Secondary Index (GSI)

The GSI is stored separately, potentially on different physical partitions.

Updates to a GSI happen asynchronously — the system eventually propagates changes from the base table.

Thus:

  - Only eventually consistent reads are possible.
  - A read might momentarily see stale data (e.g., missing or outdated items).

Streams

Optional feature that captures data modification events in DynamoDB tables. The data about these events appears in the stream in near-real time, and in the order that the events occurred.

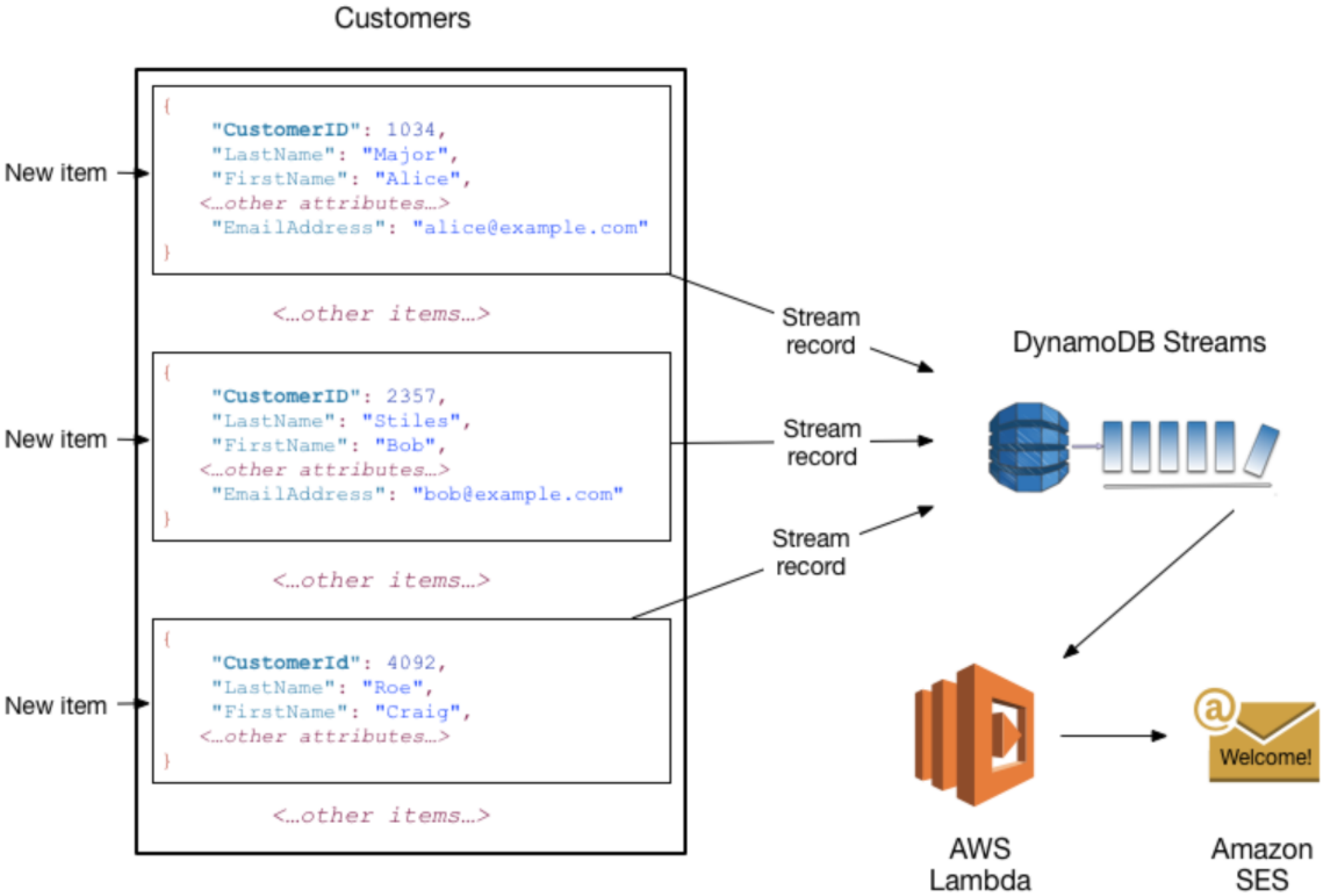
Each event is represented by a stream record. If stream is enabled on a table, Streams writes a stream record whenever one of the following events occurs:

- A new item is added to the table: The stream captures an image of the entire item, including all of its attributes.
- An item is updated: The stream captures the "before" and "after" image of any attributes that were modified in the item.
- An item is deleted from the table: The stream captures an image of the entire item before it was deleted.

Streams

Each stream record also contains the name of the table, the event timestamp, and other metadata. Stream records have a lifetime of 24 hours; after that, they are automatically removed from the stream.

Can be used together with AWS Lambda to create a trigger—code that runs automatically whenever an event of interest appears in a stream. For example, consider a Customers table that contains customer information for a company. When sending a "welcome" email to each new customer and stream is enabled on that table, the stream can be associated with a Lambda function that would run whenever a new stream record appears, but only process new items added to the Customers table. For any item that has an EmailAddress attribute, the Lambda function would invoke Amazon Simple Email Service (Amazon SES) to send an email to that address.


Customers
New item
{
"CustomerID": 1034,
"LastName": "Major",
"FirstName": "Alice",
<…other attributes…>
"EmailAddress": "alice@example.com"
}
<…other items…>
New item
{
"CustomerID": 2357,
"LastName": "Stiles",
"FirstName": "Bob",
<…other attributes…>
"EmailAddress": "bob@example.com"
}
<…other items…>
New item
{
"CustomerId": 4092,
"LastName": "Roe",
"FirstName": "Craig",
<…other attributes…>
}
<…other items…>
Stream record
Stream record
Stream record
DynamoDB Streams
AWS Lambda
Welcome!
Amazon SES

API

Working with Amazon DynamoDB from an application can be done using a few simple API operations; they are organized in categories:

- Control plane
- Data plane
- DynamoDB Streams
- Transactions

Control plane

Control plane operations are used to create and manage DynamoDB tables, work with indexes, streams, and other objects that are dependent on tables.

CreateTable – Creates a new table. Optionally, can be used to create one or more secondary indexes, and enable DynamoDB Streams for the table.

DescribeTable– Returns information about a table, such as its primary key schema, throughput settings, and index information.

ListTables – Returns the names of all of your tables in a list.

UpdateTable – Modifies the settings of a table or its indexes, creates or removes new indexes on a table, or modifies DynamoDB Streams settings for a table.

DeleteTable – Removes a table and all of its dependent objects from DynamoDB.

Data plane

Allows create, read, update, and delete (also called CRUD) actions on data in a table. Some of the data plane operations allow reading data from a secondary index.

PartiQL - a SQL-compatible query language for Amazon DynamoDB, allows to perform these CRUD operations; they can also be done via DynamoDB's classic CRUD APIs that separates each operation into a distinct API call.

ExecuteStatement – Reads multiple items from a table. Can be used to write or update a single item from a table. When writing or updating a single item, the primary key attributes must be specified.

BatchExecuteStatement – Writes, updates or reads multiple items from a table. This is more efficient than ExecuteStatement because application only needs a single network round trip to write or read the items.

Data Plane

Classic API

- Creating data
  - PutItem – Writes a single item to a table; the primary key attributes must be specified, other attributes are not mandatory.
  - BatchWriteItem – Writes up to 25 items to a table. This is more efficient than calling PutItem multiple times because the application only needs a single network round trip to write the items.
- Reading data
  - GetItem – Retrieves a single item from a table; specify the primary key for the item to find. Can retrieve the entire item, or just a subset of its attributes.
  - BatchGetItem – Retrieves up to 100 items from one or more tables. This is more efficient than calling GetItem multiple times because the application only needs a single network round trip to read the items.
  - Query – Retrieves all items that have a specific partition key; must specify the partition key value. Can retrieve entire items, or just a subset of their attributes. Optionally, apply a condition to the sort key values to only retrieve a subset of the data that has the same partition key. Can be used on a table, provided that the table has both a partition key and a sort key. Can be uses on an index, provided that the index has both a partition key and a sort key.
  - Scan – Retrieves all items in the specified table or index. Can retrieve entire items, or just a subset of their attributes. Optionally, can apply a filtering condition to return only the values that are of interest and discard the rest.

Data Plane

Classic API

- Updating data
  - UpdateItem – Modifies one or more attributes in an item; must specify the primary key for the item to modify. Can add new attributes and modify or remove existing attributes. Can also perform conditional updates, so that the update is only successful when a user-defined condition is met. Optionally, can implement an atomic counter, which increments or decrements a numeric attribute without interfering with other write requests.
- Deleting data
  - DeleteItem – Deletes a single item from a table; must specify the primary key for the item that is to be deleted.
  - BatchWriteItem – Deletes up to 25 items from one or more tables. This is more efficient than calling DeleteItem multiple times because the application only needs a single network round trip to delete the items.

Streams

DynamoDB Streams operations are used to disable a stream on a table, and allow access to the data modification records contained in a stream.

ListStreams – Returns a list of all the streams, or just the stream for a specific table.

DescribeStream – Returns information about a stream, such as its Amazon Resource Name (ARN) and where the application can begin reading the first few stream records.

GetShardIterator – Returns a shard iterator, which is a data structure that the application uses to retrieve the records from the stream.

GetRecords – Retrieves one or more stream records, using a given shard iterator.